



**INSTITUTO POLITÉCNICO NACIONAL.
UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**



**Unidad de aprendizaje:
Bases de datos distribuidas.**

Tarea de arquitectura y tipos de gestores de bases de datos distribuidas

Alumnos:

- Fuentes García Liliana Alejandra.

Profesora:

Carlos De La Cruz Sosa.

Fecha de entrega:

20 de marzo del 2026.

Tema 1. Arquitecturas Distribuidas de BD

➤ Definición y Transparencias.

Una base de datos distribuida es un conjunto de datos lógicamente relacionados almacenados en múltiples sitios interconectados. Un *DDBMS* gestiona esos datos y “hace transparente” la distribución al usuario, manejando fragmentación y replicación internamente. Según Özsu & Valduriez, las BD distribuidas se caracterizan por: datos fragmentados, posibles réplicas, sitios bajo control local (autonomía) y participación en aplicaciones globales. Existen distintas “dimensiones” para clasificar DDBMS: distribución (cómo se reparte el dato físico), autonomía (qué control tienen sitios individuales) y heterogeneidad (diferencias en modelos o SGBD). Por ejemplo, en un sistema homogéneo todos los nodos usan el mismo SGBD y hay un esquema global. En uno heterogéneo, podría existir un esquema federado (único esquema global, autonomía mínima) o multi-base (sin esquema global, esquema construido dinámicamente). En resumen, la transparencia de distribución, fragmentación y réplica permite a los usuarios interactuar con la BD como si fuera única, ocultando su dispersión física.

➤ Principales arquitecturas.

Las arquitecturas físicas más comunes incluyen:

- **Cliente-Servidor:** uno o varios servidores centrales que alojan la base de datos y clientes que les envían consultas. Puede ser *master-esclavo* (un servidor maestro principal) o *multi-maestro*.
- **Peer-to-Peer (Igual a Igual):** todos los nodos son equivalentes, pueden recibir consultas y comunicarse mutuamente. No hay nodo central. Este modelo evita puntos únicos de fallo y cuellos de botella, y es típico de sistemas NoSQL como Cassandra.
- **Compartida (Shared) vs No compartida (Shared-nothing):**
 - *Shared-Memory:* pocos procesadores comparten memoria común; típico en bases de datos paralelas internas a un servidor.
 - **Shared-Disk:** múltiples procesadores/instancias acceden a un mismo almacenamiento (ej. SAN/NFS). Oracle RAC es un ejemplo: varias instancias de Oracle BD leen/escriben un disco común.
 - **Shared-Nothing:** cada nodo tiene CPU, RAM y disco propio, no comparte nada. Es el esquema más común en BDD actuales: Cassandra, Greenplum, etc. El Shared-Nothing elimina puntos únicos de fallo y escala linealmente agregando nodos, particionando (shardeando) los datos.

```

graph LR
    SN[Shared-Nothing] -- "Cada nodo independiente" --> SNchar(No comparte disco/almac.)
    SN -- "Ejemplos: Cassandra, RedShift" --> SNex
    SD[Shared-Disk] -- "Disco compartido entre nodos" --> SDchar(Comparten unidad de almacenamiento)
    SD -- "Ejemplo: Oracle RAC" --> SDex
    SM[Shared-Memory] -- "Memoria compartida entre nodos" --> SMchar(Comparten RAM)
    SM -- "Ejemplo: Sistemas paralelos" --> SMex

```

Ilustración 1. Esquema comparativo de arquitecturas shared-nothing, shared-disk y shared-memory. En shared-nothing cada servidor opera con su propio CPU, RAM y disco, mientras que shared-disk y shared-memory implican recursos compartidos.

➤ **Distributed vs Parallel.**

En una BD distribuida los nodos pueden estar geográficamente dispersos (red WAN) y la comunicación es costosa. En contraste, en sistemas paralelos (data warehouses) los nodos suelen estar próximos (en un clúster de alta velocidad). Oracle RAC combina ambos enfoques: ejecuta *funciones shipping* (enviar consulta) para análisis paralelos, pero para transacciones OLTP suele usar *data shipping* local (caché de datos) para evitar overhead de commit distribuido.

➤ **Fragmentación de datos.**

La fragmentación divide tablas en partes lógicas almacenadas en diferentes sitios, manteniendo la capacidad de reconstituir la tabla completa. Existen tres tipos básicos:

- **Fragmentación Horizontal:** cada fragmento contiene un subconjunto de filas seleccionadas por un predicado (por ejemplo, rango de claves).
- **Fragmentación Vertical:** cada fragmento contiene un subconjunto de columnas (atributos), incluyendo la clave primaria para permitir reconstruir la tabla por *join*.
- **Fragmentación Mixta (híbrida):** combinación de ambas (ej. fragmentar primero por columnas y luego por filas sobre esa tabla resultante).

La fragmentación mejora la localización de datos y el rendimiento, pero añade complejidad de reconstrucción (suma o *join* de fragmentos). Los gestores distribuidos usan este mecanismo para distribuir carga (por ejemplo, cada región almacena sus clientes locales) o para escalabilidad horizontal.

➤ Replicación de datos.

La replicación consiste en copiar fragmentos (o tablas completas) en múltiples nodos, protegiendo contra fallos y mejorando disponibilidad. Hay dos modos generales:

- **Replicación Sincrónica:** la operación de escritura espera confirmación de múltiples réplicas (p.ej. mayoría) antes de considerarse completada. Garantiza **consistencia fuerte** inmediata, pero puede elevar la latencia.
- **Replicación Asíncrona:** la escritura retorna éxito después de aplicar localmente, y se propaga a réplicas posteriormente. Mejora el rendimiento pero introduce *lag*, resultando en consistencia eventual o ajustable. P.ej., Cassandra es asíncrono por defecto, aunque su factor de réplica y nivel de consistencia son configurables por consulta. Spanner y CockroachDB prefieren la vía síncrona (consenso Paxos/Raft) para dar consistencia fuerte global.

➤ Modelos de consistencia.

Los sistemas distribuidos afrontan el trade-off CAP (consistencia, disponibilidad, particiones). Algunos priorizan disponibilidad y particiones (Cassandra, MongoDB en su modo eventual) admitiendo lecturas obsoletas; otros buscan consistencia fuerte (Spanner, Cockroach). Cassandra soporta *consistencia ajustable*: con factorización de réplica RF=3 y nivel QUORUM, necesita confirmación de 2 nodos para leer/escribir. Google Spanner garantiza *consistencia externa* mediante su reloj global (TrueTime) y réplicas Paxos síncronas: en cada conjunto de réplicas hay un líder para escrituras y réplicas lectoras, pudiendo leer siempre datos actualizados.

➤ Concurrencia y recuperación.

En DB distribuidas las transacciones multi-sitio requieren protocolos complejos. Suele emplearse *2-Phase Commit (2PC)* para asegurar atomicidad global: el coordinador solicita prepare/commit a cada sitio. Esto, junto con 2PL distribuido u otros mecanismos (timestamp, MVCC), mantiene la coherencia ACID pero incrementa latencia y riesgo de bloqueos distribuidos. En caso de fallo de un nodo durante commit, el sistema debe usar logs de transacción locales para recuperar o abortar transacciones globales. Los sistemas modernos combinan replicación de logs y checkpoints distribuidos para fiabilidad; por ejemplo, si un nodo replica en 3 sitios y uno falla, las réplicas sobrevivientes pueden rehacer operaciones pendientes.

➤ Optimización de consultas distribuidas.

El optimizador de un DDBMS decide si llevar la consulta al dato o viceversa. Estrategias típicas: enviar subconsultas (procedimientos almacenados o filtros) a nodos remotos (*query shipping*), o traer tablas remotas al coordinador para procesar localmente (*data shipping*). Por ejemplo, Oracle RAC paraleliza automáticamente grandes consultas OLAP rompiéndolas en fragmentos SQL ejecutados en cada instancia. Las redes lentas favorecen minimizar el tráfico moviendo operaciones a

los datos; en cambio, clusters de alta velocidad pueden enviar datos intermedios. El costo de envío y las estadísticas globales guían estas decisiones.

➤ **Seguridad.**

Las BD distribuidas refuerzan la seguridad usando cifrado (TLS) en la comunicación entre nodos y con clientes, autentican usuarios en cada instancia y aplican control de acceso granular local. También pueden usar VPN o redes privadas para aislar nodos. La heterogeneidad añade reto: se necesitan mecanismos coherentes de autorización en múltiples sistemas.

Tema 2. Gestores Distribuidos: Arquitectura y Comparativa

A continuación, se analizan varios SGBD distribuidos prominentes. Para cada uno se describe su arquitectura, soporte de fragmentación y replicación, consistencia, escalabilidad, tolerancia a fallos, licenciamiento y casos de uso típicos.

➤ **PostgreSQL BDR (Postgres-BDR).**

- **Arquitectura:** Es una extensión de PostgreSQL (EnterpriseDB) que habilita *multi-maestro* con topología de red mallada (cada nodo se conecta a los demás). Es decir, cualquier nodo puede recibir escrituras y las propaga a todos los demás (“multi-master logical replication”). No hay esquema central: todos los nodos forman un clúster homogéneo. BDR configura por defecto replicación asíncrona por rows, pero permite opciones de semisincronicidad.
- **Fragmentación:** BDR es principalmente replicación. Existe una funcionalidad llamada *AutoPartition* que permite fragmentar tablas horizontalmente entre nodos (ataque un sharding básico), pero típicamente se usa la partición nativa de PostgreSQL o configuraciones manuales de fragmentos. No es un enfoque de base datos nativa para shard; la distribución de datos suele manejarse a nivel de aplicación o con herramientas externas. En esencia, BDR asume réplicas completas de las tablas entre todos los nodos.
- **Replicación:** Multi-maestro sincrónica/asíncrona. Por defecto es asíncrona (un nodo hace commit local, luego reenvía los cambios), aunque EDB provee opciones semisincrónicas (configurable). Se logra alta disponibilidad y balanceo de carga: fallos de un nodo no detienen al clúster (los demás siguen recibiendo cambios).
- **Consistencia:** Al ser multi-maestro debe manejar conflictos. BDR incluye mecanismos avanzados de resolución de conflictos. Dependiendo de la configuración, puede ofrecer consistencia eventual o casi fuerte. En la práctica, se garantiza que cada transacción es aplicada a todos los nodos, pero el orden relativo de transacciones concurrentes debe sincronizarse (vía vectores de versión). El modelo resultante puede considerarse “convergente” (eventual

cuando todos los cambios se propagan) con opciones de aislamiento de transacción estándar (ACID en cada nodo).

- **Ventajas/Desventajas:** Permite escalabilidad lectora y escritura multi-sitio, tolerancia a fallos y sin cuello de botella maestro único. Sin embargo, introduce sobrecarga de control de conflictos y réplica, y requiere red de baja latencia entre nodos para evitar retrasos. No es tan simple como la replicación maestra-esclavo tradicional. Es ideal para ambientes con cargas moderadas y necesidad de escritura en varias ubicaciones.
- **Casos de uso:** Geodistribución de aplicaciones empresariales, multi-master en tiendas retail, entornos multiregionales que requieren escritura simultánea. También se usa para balanceo de lectura/escritura. Por ejemplo, sistemas ERP que operan en varias oficinas.
- **Infraestructura:** Clúster de servidores PostgreSQL con la extensión BDR instalada en cada uno. Se recomienda hardware homogéneo, buena red (mínima latencia) y suficiente RAM para caches. No hay requerimientos especiales aparte de lo usual en Postgres; escalado horizontal añadiendo nodos.
- **Licencia:** El core de PostgreSQL es BSD (open source) pero BDR es ofrecido por EnterpriseDB bajo licencia comercial. Existe una versión Community Free (EDB PostgreSQL Distribuido) con limitaciones, y versiones Enterprise con soporte comercial.

➤ **MySQL Group Replication.**

- **Arquitectura:** Plugin de MySQL 8+ que proporciona replicación *multi-primaria* basada en consenso Paxos. Los nodos se unen a un grupo; en *modo single-primary*, sólo uno escribe a la vez (auto-elección), y en *modo multi-primary* todos pueden escribir concurrentemente. Internamente, utiliza un protocolo transaccional que combina binlogs de MySQL con certificación de transacciones para evitar conflictos, y un subsistema de grupo (XCom) para comunicación fiable.
- **Fragmentación:** No tiene sharding nativo. Cada miembro del grupo replica todo el conjunto de datos. No es un SGBD *sharded*, sino *replicado*. Para carga horizontal se suele usar MySQL InnoDB Cluster o arquitecturas externas (p. ej. MySQL NDB, Vitess) en paralelo.
- **Replicación:** Casi síncrona. Usa 2PC y Paxos: todas las transacciones se propagan a todos los nodos del grupo; la decisión de commit requiere certificar que no hay conflicto de escritura. En multi-primary, cada commit se envía al grupo y se aplica local y remotamente antes de confirmar, alcanzando alta consistencia (pues debe recibir la mayoría). En single-primary, al menos 2 nodos deben confirmar el registro en el log (semisincrónico). En resumen, ofrece *consistencia fuerte* (el estado del grupo

es unificado) y tolerancia a fallos mientras haya nodos mayoritarios operativos.

- **Consistencia:** Totalmente consistente según el protocolo: cada transacción se ordena globalmente. A nivel de aplicación se garantiza que no hay divergencias entre réplicas (certificación asegura que todas aceptan o ninguna). En caso de partición, MySQL detiene el grupo hasta resolver quorum (no sacrifica consistencia).
- **Ventajas/Desventajas:** Alta disponibilidad con verdadero multi-master. Automática reconfiguración al caer un nodo, auto-elección de primarios. Sin embargo, la latencia de commit es mayor que en replicación asíncrona, y el rendimiento de escritura se ve limitado por latencia de consenso. Se sugiere usar pocos nodos (3-5) y buena red. Conflictos se evitan vía bloqueos globales, pero hay sobrecarga de control.
- **Casos de uso:** Aplicaciones transaccionales que requieren alta disponibilidad y tolerancia a fallos, con escrituras en múltiples sitios (p.ej. sistemas financieros globales, o aplicaciones críticas web). No es ideal para consulta analítica masiva.
- **Infraestructura:** Clúster MySQL InnoDB sobre red confiable (p. ej. LAN rápida). Cada nodo corre MySQL 8+ con plugin GR activo. Se recomienda configuraciones N+1 (backup) y sistemas de monitoreo. No requiere hardware especial, pero se beneficia de discos rápidos (para logs) y latencia reducida.
- **Licencia:** GPL v2 (MySQL es open source). El plugin Group Replication es parte de la edición comunitaria de MySQL.

➤ Oracle Real Application Clusters (RAC).

- **Arquitectura:** Implementa un clúster *shared-disk* de Oracle Database. Varias instancias de Oracle BD corren en distintos servidores, pero todas comparten el mismo almacenamiento (Storage Area Network). Gracias a Oracle Clusterware, las instancias coordinan sus caches de datos (Cache Fusion) permitiendo acceso concurrente a todos los datos. En esencia cada instancia cree ver todo el DB, y Oracle sincroniza cachés a nivel de bloque. No es P2P puro (pues comparten disco), ni *shared-nothing* puro (cada inst. lee/escribe localmente caches).
- **Fragmentación:** No aplica fragmentación lógica en RAC. Todos los nodos acceden al mismo conjunto de tablas. Oracle puede particionar internamente tablas (particionamiento horizontal), pero eso es un esquema interno de datos, no fragmentación distribuida propiamente dicha. RAC no sharea datos por nodo; su foco está en paralelizar el acceso dentro de un único BD global.

- **Replicación:** La única “replicación” en RAC es la sincronización de cachés: cuando la instancia A escribe un bloque, Oracle lo invalida en las demás y lo propaga por red (Cache Fusion). No hay replicación tipo maestro-esclavo; todas las instancias escriben a la misma BD. En contraste, Oracle ofrece además soluciones de replicación (Data Guard, GoldenGate) fuera del ámbito de RAC para DR o geo-replicación, pero RAC en sí usa almacenamiento compartido, por lo que no hay copia de datos separada.
- **Consistencia:** Consistencia transaccional ACID fuerte. RAC funciona como un único sistema transaccional distribuido: todas las instancias participan en el mismo espacio de datos, usando bloqueos distribuidos y transacciones 2PC internas. No sufre de *eventual consistency*, dado que cada transacción observa un snapshot consistente.
- **Escalabilidad:** Escala principalmente lecturas. Aumentar instancias mejora throughput lecturas y permite balanceo de carga de sesiones. Para escrituras, se mejora moderadamente: el procesamiento se paraleliza, pero todas las instancias compiten por el mismo disco, así que hay un límite de escala. En práctica, RAC escala bien OLTP mediano (varios cientos de instancias, si hardware y almacenamiento son potentes), pero no escala infinitamente como un sistema P2P (ver [51†L198-L207]).
- **Tolerancia a fallos:** Muy alta. Si una instancia o nodo falla, las demás continúan sirviendo (por almacenamiento compartido). Las transacciones activas en el nodo caído se revierten, y un nuevo nodo puede tomar su lugar. Necesita votación de quorum para sobrevivir a fallos: Oracle Clusterware usa métodos Paxos-like para acordar estado del clúster. En sumario, no hay SPoF en CPU (todo BD replicado en disco), aunque el almacenamiento puede ser único punto de fallo salvo configuraciones redundantes.
- **Ventajas/Desventajas:** Permite continuidad 24/7 y crecimiento incremental, además de tolerar fallos de servidor. Ideal para workloads mixtos (OLTP+OLAP) con alta demanda. La complejidad es grande: requiere hardware costoso (almacenamiento compartido de alto rendimiento, interconexión rápida) y experiencia operacional. Costo de licencia/licencia de RAC es muy alto (Oracle Database Enterprise Edition con opción RAC).
- **Casos de uso:** Grandes empresas con cargas críticas OLTP/OLAP 24x7 (bancos, telcos, ERP globales) que necesitan escalado de transacciones y alta disponibilidad. También Data Warehouses en exadata suelen usar RAC.
- **Infraestructura:** Servidores con Linux/Unix, interconectados por red privada de baja latencia. SAN/NAS compartido (o Exadata, Oracle ASM). Clusterware + discos redundantes. Esquema tradicional on-premise; Oracle Cloud ofrece también RAC.
- **Licencia:** Propietaria. Requiere Oracle Database Enterprise Edition y la opción RAC (altos costos). No es open source.

➤ **Oracle Real Application Clusters (RAC).**

- **Arquitectura:** Implementa un clúster *shared-disk* de Oracle Database. Varias instancias de Oracle BD corren en distintos servidores, pero todas comparten el mismo almacenamiento (Storage Area Network). Gracias a Oracle Clusterware, las instancias coordinan sus caches de datos (Cache Fusion) permitiendo acceso concurrente a todos los datos. En esencia cada instancia cree ver todo el DB, y Oracle sincroniza cachés a nivel de bloque. No es P2P puro (pues comparten disco), ni *shared-nothing* puro (cada inst. lee/escribe localmente caches).
- **Fragmentación:** No aplica fragmentación lógica en RAC. Todos los nodos acceden al mismo conjunto de tablas. Oracle puede particionar internamente tablas (particionamiento horizontal), pero eso es un esquema interno de datos, no fragmentación distribuida propiamente dicha. RAC no *sharea* datos por nodo; su foco está en paralelizar el acceso dentro de un único BD global.
- **Replicación:** La única “replicación” en RAC es la sincronización de cachés: cuando la instancia A escribe un bloque, Oracle lo invalida en las demás y lo propaga por red (Cache Fusion). No hay replicación tipo maestro-esclavo; todas las instancias escriben a la misma BD. En contraste, Oracle ofrece además soluciones de replicación (Data Guard, GoldenGate) fuera del ámbito de RAC para DR o geo-replicación, pero RAC en sí usa almacenamiento compartido, por lo que no hay copia de datos separada.
- **Consistencia:** Consistencia transaccional ACID fuerte. RAC funciona como un único sistema transaccional distribuido: todas las instancias participan en el mismo espacio de datos, usando bloqueos distribuidos y transacciones 2PC internas. No sufre de *eventual consistency*, dado que cada transacción observa un snapshot consistente.
- **Escalabilidad:** Escala principalmente lecturas. Aumentar instancias mejora throughput lecturas y permite balanceo de carga de sesiones. Para escrituras, se mejora moderadamente: el procesamiento se paraleliza, pero todas las instancias compiten por el mismo disco, así que hay un límite de escala. En práctica, RAC escala bien OLTP mediano (varios cientos de instancias, si hardware y almacenamiento son potentes), pero no escala infinitamente como un sistema P2P (ver [51†L198-L207]).
- **Tolerancia a fallos:** Muy alta. Si una instancia o nodo falla, las demás continúan sirviendo (por almacenamiento compartido). Las transacciones activas en el nodo caído se revierten, y un nuevo nodo puede tomar su lugar. Necesita votación de quorum para sobrevivir a fallos: Oracle Clusterware usa métodos Paxos-like para acordar estado del clúster. En sumario, no hay SPoF en CPU (todo BD replicado en disco), aunque el almacenamiento puede ser único punto de fallo salvo configuraciones redundantes.

- **Ventajas/Desventajas:** Permite continuidad 24/7 y crecimiento incremental, además de tolerar fallos de servidor. Ideal para workloads mixtos (OLTP+OLAP) con alta demanda. La complejidad es grande: requiere hardware costoso (almacenamiento compartido de alto rendimiento, interconexión rápida) y experiencia operacional. Costo de licencia/licencia de RAC es muy alto (Oracle Database Enterprise Edition con opción RAC).
- **Casos de uso:** Grandes empresas con cargas críticas OLTP/OLAP 24x7 (bancos, telcos, ERP globales) que necesitan escalado de transacciones y alta disponibilidad. También Data Warehouses en exadata suelen usar RAC.
- **Infraestructura:** Servidores con Linux/Unix, interconectados por red privada de baja latencia. SAN/NAS compartido (o Exadata, Oracle ASM). Clusterware + discos redundantes. Esquema tradicional on-premise; Oracle Cloud ofrece también RAC.
- **Licencia:** Propietaria. Requiere Oracle Database Enterprise Edition y la opción RAC (altos costos). No es open source.

➤ Google Cloud Spanner

- **Arquitectura:** Servicio de BD relacional global de Google Cloud. Combina almacenamiento distribuido y consenso síncrono. Internamente, cada base de datos se divide en **divisiones (splits)** ordenadas por clave primaria. Estas divisiones se guardan en un sistema de archivos distribuido (Colossus) que replica bytes automáticamente. Cada división se replica de forma *alineada con Paxos*: un grupo de réplicas (usualmente 3-5) gestiona cada división. Una réplica actúa líder para escrituras, mientras las demás sirven lecturas. Usa *TrueTime* (reloj global) para lograr consistencia externa en transacciones.
- **Fragmentación:** Implementa **particionamiento horizontal** automático. Las divisiones son esencialmente shards basados en rangos de llave. Spanner ajusta dinámicamente las fronteras de división según el tamaño y la carga. El usuario no gestiona las particiones (ninguna clave de shard que definir): Google hace *auto-sharding* continuo. Esto permite escalar bases inmensas sin puntos de bloqueo de metadatos.
- **Replicación: Síncrona global.** Cada escritura pasa por consenso Paxos: antes de confirmar, se requiere la mayoría de réplicas de ese grupo voten a favor. Gracias al soporte de Colossus, las mutaciones de BD se escriben en archivos replicados por debajo, y adicionalmente Spanner mantiene sus propias réplicas lógicas para disponibilidad/regionalidad. De modo que cada división tiene típicamente una réplica en varias zonas geográficas. En cada conjunto Paxos, un líder procesa escrituras, y réplicas voto. Los clientes pueden leer desde réplicas de lectura, pero para lecturas fuertes suelen contactar al líder para asegurar no obsoletas.

- **Consistencia:** Consistencia fuerte global (relacional ACID). El uso de Paxos y TrueTime permite que cualquier lectura dentro de una transacción vea todos los commits anteriores hasta un marcador de tiempo coherente. De hecho, Spanner ofrece *external consistency*: una global snapshot linealizable.
- **Escalabilidad:** Muy alta. Diseñado para bases de datos colosales (billones de filas) y alta carga, escalando automática o manualmente por regiones y nodos. Cada división es servida por varias réplicas, y se pueden añadir nuevas réplicas regionales para mayor performance local. Google garantiza que la escalabilidad puede crecer según demanda (sin límite técnico conocido) ya que redistribuye datos y cargas bajo el capó.
- **Tolerancia a fallos:** Extrema. Con réplicas en múltiples zonas/regiones, Spanner puede perder réplicas sin pérdida de servicio (una mayoría de réplicas es suficiente para continuar). Por ejemplo, con 5 réplicas en 5 zonas, puede tolerar la caída de hasta 2 simultáneas. Además, Colossus se encarga de recuperación a nivel de bloque. Spanner es un servicio administrado que replica datos geográficamente.
- **Ventajas/Desventajas:** Ofrece un RDBMS distribuido totalmente administrado con consistencia fuerte y SQL familiar. Su diseño global es único. El “lado malo” es la latencia (coordinar réplicas Paxos globalmente añade overhead) y el precio (es un servicio pago por uso). No es auto-hospedable (solo en Google Cloud) y es de uso más complejo comparado a BD locales.
- **Casos de uso:** Bases de datos globales que requieren transacciones ACID a escala planetaria. Ej: sistemas bancarios multinacionales, plataformas de e-commerce global, catálogos distribuidos. También para replicar cargas de SQL tradicionales en múltiples regiones.
- **Licencia/Acceso:** Es un servicio Cloud (Google Cloud Spanner); no hay licencia de software independiente, solo pago por uso (instancia+almacenamiento).

Referencias

- Özsu, M. T., & Valduriez, P. (2011). *Principles of Distributed Database Systems* (3ª ed.). Springer. (Conceptos de DDBMS y arquitecturas básicas).
- Oracle Corporation. *Real Application Clusters (RAC)*. Página oficial (2023).
- EnterpriseDB. *EDB Postgres Distributed (BDR) Documentation* (v4.3.8, 2023)..
- MySQL (Oracle). *MySQL Group Replication – Manual de Referencia 8.4* (2025).
- Datastax/Apache. *Apache Cassandra Documentation* (latest, 2025). Página de conceptos.
- Krantz, T. y Jonker, A. *¿Qué es Apache Cassandra?* (IBM Think, 2024).
- MongoDB Inc. *Manual de MongoDB – Sharding y Replica Sets* (2026).
- Cockroach Labs. *CockroachDB Architecture Overview* (v25-v26, 2025); *Licensing FAQs* (2025).
- Google Cloud. *Cloud Spanner Documentation* (2026).
- GeeksforGeeks. *Fragmentation in Distributed DBMS* (2025).
- Sachdev, E. et al. *Oracle RAC – The New Generation* (O'Reilly/Oracle Whitepaper, 2024).